21422018

CSY3024 (Databases 3)

Level 6

Assignment 1

(First-sit, 2022/23)

Module Leader: Santosh Khanal

Email: santosh.khanal@nami.edu.np

Table of Content

Diagram List

Analysis of the dataset

The "EPL_matches.csv" file, which contains all the information pertinent to the database, was made available. A Key to Results Data and Match Statics was constructed to clarify all the acronyms in the headers so that users may better understand how the database is going to be organized.

Key to results data:

Season = League season

DateTime - Date and time of the matches

HomeTeam = Home Team

AwayTeam = Away Team

FTHG and HG = Full Time Home Team Goals

FTAG and AG = Full Time Away Team Goals

FTR and Res = Full Time Result (H=Home Win, D=Draw, A=Away Win)

HTHG = Half Time Home Team Goals

HTAG = Half Time Away Team Goals

HTR = Half Time Result (H=Home Win, D=Draw, A=Away Win)

Match Statistics (where available)

Attendance = Crowd Attendance

Referee = Match Referee

HS = Home Team Shots

AS = Away Team Shots

HST = Home Team Shots on Target

AST = Away Team Shots on Target

HHW = Home Team Hit Woodwork

AHW = Away Team Hit Woodwork

HC = Home Team Corners

AC = Away Team Corners

HF = Home Team Fouls Committed

AF = Away Team Fouls Committed

HFKC = Home Team Free Kicks Conceded

AFKC = Away Team Free Kicks Conceded

HO = Home Team Offsides

AO = Away Team Offsides

HY = Home Team Yellow Cards

AY = Away Team Yellow Cards

HR = Home Team Red Cards

AR = Away Team Red Cards

HBP = Home Team Bookings Points (10 = yellow, 25 = red)

ABP = Away Team Bookings Points (10 = yellow, 25 = red)

The key to results data gives details on the various statistics pertaining to a football game, such as the season, date, and time of the game, home team, away team, full-time and half-time goals scored by each team, full-time result (Home Win, Draw, or Away Win), half-time result, attendance, referee, shots taken by each team, shots on target by each team, corners taken by each team, fouls committed by each team, free kicks conceded by each team.

It is clear that the initial terms have something to do with match statistics (such as the teams that will be competing and goals scored, etc.).

The following attributes are required for the queries:

[Date, HomeTeam, AwayTeam, FTHG, FTAG, HTHG, HTAG, FTR, HTR, Referee, HS, AS].

Design of the Graph Model

In the original design, the relationships that would store all the match statistics were expected to come after the team nodes with the team names.

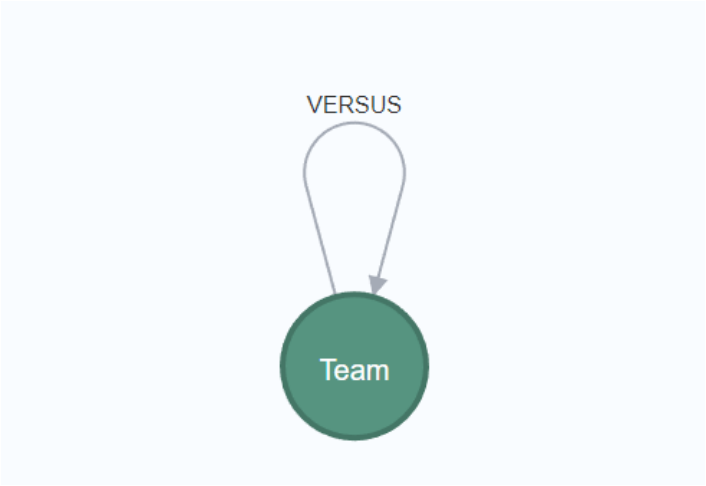

Fig.1. First schema

Although this concept would be relatively straightforward, the questions would be difficult. Complex queries are challenging to execute with simple models.

It was decided to establish a node that would store the match statistics rather than having the relationship record every single match statistic. The data would be more skewed in such cases. As a result, the relationships would either be "HOME" or "AWAY" depending on whether the team would be playing as the home team or the away team in that game.

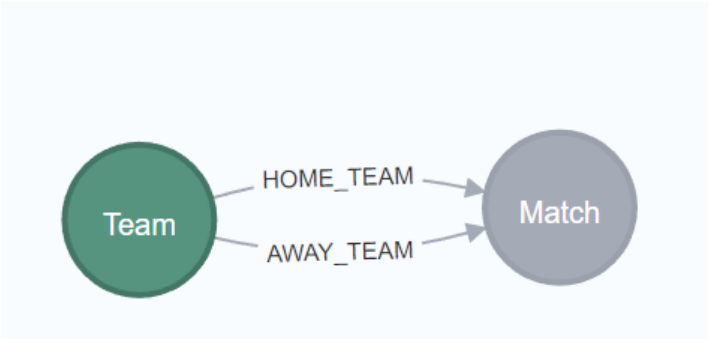

Fig.2. Second schema

The new schema improves query ease by addressing a few issues, but there is still room for improvement. Instead of having full-time home team goals and full-time away team goals, it would be preferable to just keep full-time goals in the relationship from the team to the game node. The "Referee" node was introduced since they appear frequently and it would be better if they had their own node, which might help make searches about it easier. Another good improvement would be to replace "HOME_TEAM" and "AWAY_TEAM" with "WON", "DRAW", or "LOST" in the relationship between the "TEAM" node and "MATCH" node. Knowing the result of each game directly from the relationship would make it much easier to perform queries related to game results.

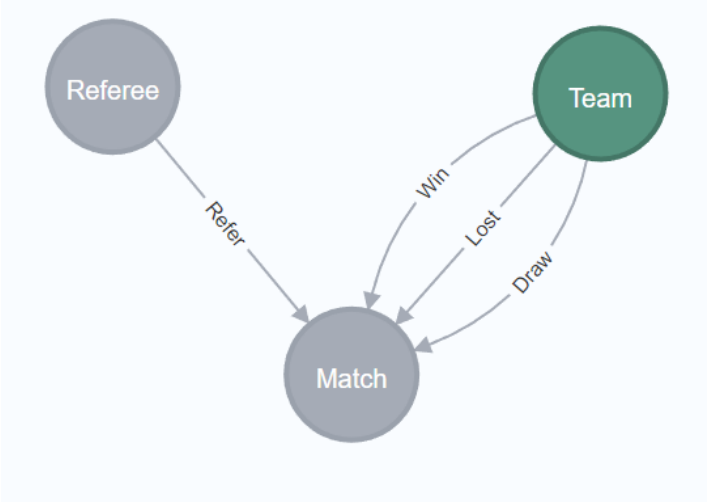

Fig.3. Final schema

Creation of the database

The following query loads data from a CSV file into a Neo4j graph database, creating nodes for each team that participated in the match and creating a relationship between them with the match statistics as properties of the relationship.

```
//LOAD CSV clause is used to read data from a CSV file

LOAD CSV WITH HEADERS FROM 'file:///EPL_matches.csv' AS row

// MERGE clause is to create unique nodes for each team based on their name

//Creates a relationship between the teams using the CREATE clause, with properties for the match statistics.

MERGE (at:Team {name: row.HomeTeam})

MERGE (ht:Team {name: row.AwayTeam})

CREATE (ht) - [:PLAYED_AGAINST {score:row.FTHG + '-' + row.FTAG,

season : row.Season,

DateTime: row.DateTime,

HG: row.FTHG,

AG: row.FTAG,

Res: row.FTR,

HTHG: row.HTHG,

HTAG: row.HTAG,

HTR: row.HTR,

Referee: row.Referee,

HS: row.HS,

AS: row.AS,

HST: row.HST,

AST: row.AST,

HC: row.HC,

AC: row.AC,

HF: row.HF,

AF: row.AF,

HY: row.HY,

AY: row.AY,
```

HR: row.HR,

AR: row.AR}]-> (at)

Answers to the queries

These are the queries that were given to solve:

1) List all teams that have ever played EPL matches since 2000.

```
MATCH (t:Team)
RETURN t.name
```

All nodes in the graph with the label "Team" are matched by the statement MATCH (t:Team) RETURN t.name, which also returns the value of the "name" attribute for each matched node. In essence, this query provides a list of all the teams that are present in the graph. The output of the query is specified by the RETURN clause.

Output:

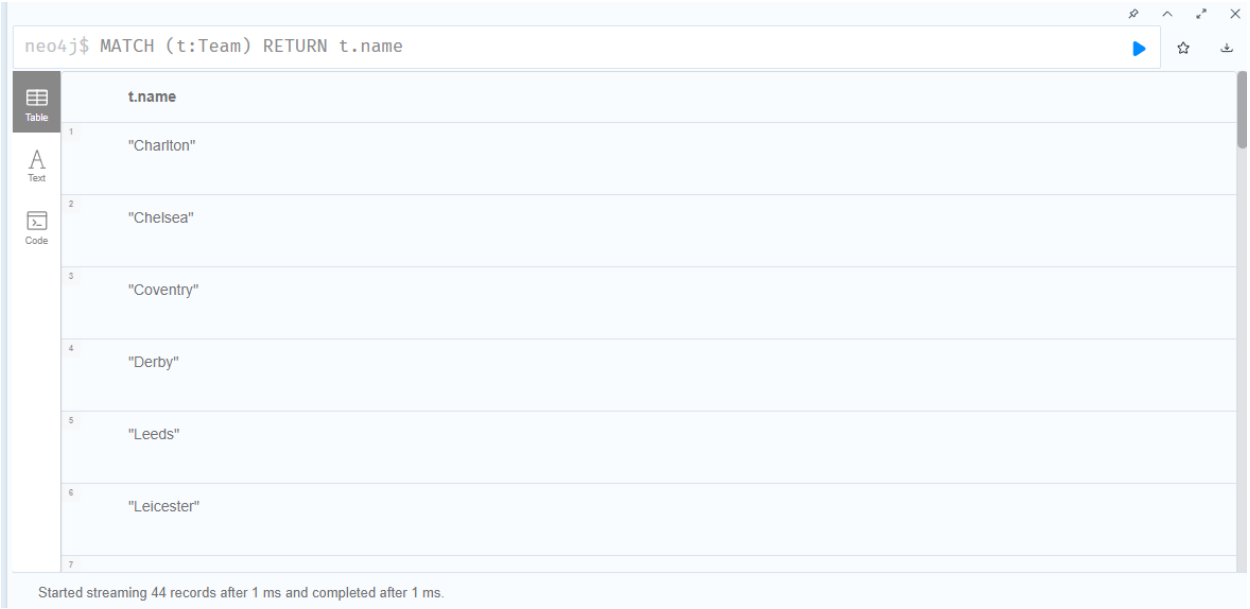

Fig.4. Display all teams that have ever played EPL matches since 2000.

2) Display all matches “Liverpool” won against “Man United” since 2010.

```
MATCH (ht:Team)-[g:PLAYED_AGAINST]->(at:Team)
WHERE (ht.name = 'Liverpool' AND at.name = 'Man United' OR ht.name = 'Man United' AND at.name
= 'Liverpool')
AND g.HG > g.AG AND g.season >= '2010-11'
RETURN COUNT(*) AS matches_won
```

First, each node with a label is matched with a team, and any relationships between those teams are labeled as PLAYED_AGAINST. Then WHERE clause filters the results based on the criteria that the names of the home team and away team must both be "Man United" or "Liverpool." Additionally, it excludes games in which the host team scored more goals than the visiting club. In the end, it counts the number of matches that meet the requirements and returns the number in the form of an alias name called "matches_won."

Output:

Fig.5. Matches "Liverpool" won against "Man United" since 2010.

3) Display the top five referees and the number of matches they have refereed since 2000.

```
MATCH (ht:Team)-[g:PLAYED_AGAINST]->(at:Team)
WHERE g.season >= '2000'
WITH g.Referee AS Referee, COUNT(*) AS matches_Refereed
ORDER BY matches_Refereed DESC
LIMIT 5
RETURN Referee, matches_Refereed
```

A relationship (g) with the label PLAYED_AGAINST is used to match the pattern where a team (ht) has faced off against another team (at). This is done first using the MATCH clause. The WHERE clause is then used to select the results based on whether the season is equal to or greater than 2000. The query groups the results by the referee's name (g.Referee) and uses the tally function to tally the number of matches they have officiated. It uses the ORDER BY clause to sort the results in decreasing order according to the number of matches it has refereed, and the LIMIT clause to restrict the results to the top five referees. Lastly, it returns the referee's name and the number of games they have officiated using the RETURN clause.

Output:

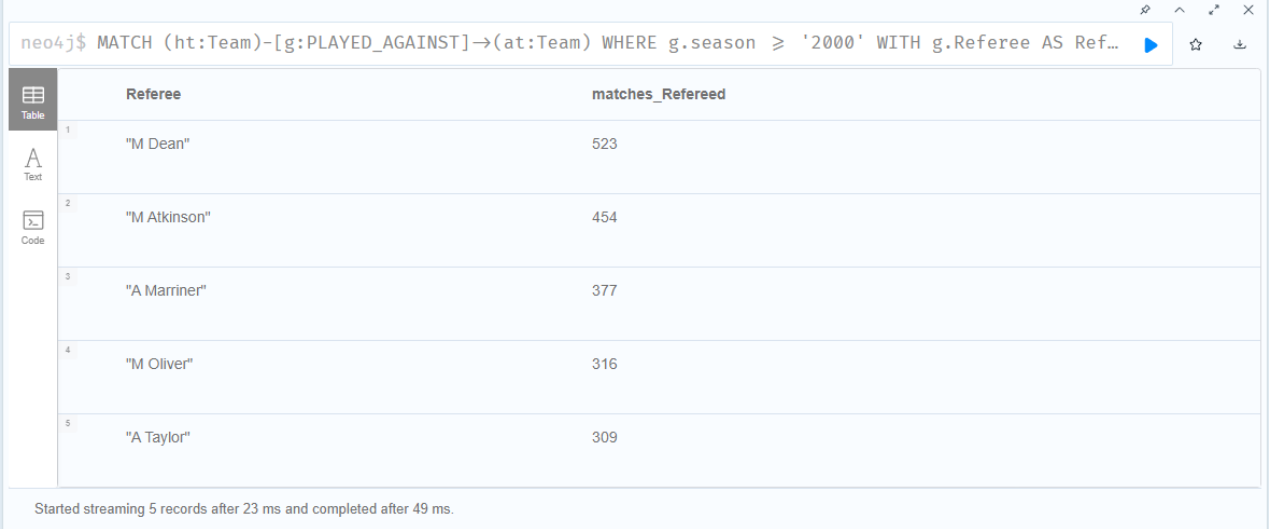

Fig.6. Top five referees and the number of matches they have refereed since 2000.

4) Display all teams and the total number of goals they scored and conceded in the 2020/21 season.

```
MATCH (t:Team)-[g:PLAYED_AGAINST]->()
WHERE g.season = '2020-21'
WITH t, SUM(toInteger(g.HG)) AS goals_scored, SUM(toInteger(g.AG)) AS goals_conceded
RETURN t.name, goals_scored, goals_conceded
```

Nodes that have a 'Team' label and a relationship of type 'PLAYED_AGAINST' to any other node are matched using the MATCH command. The relationships are filtered so that only those containing a 'season' attribute with the value '2020-21' are shown. The query then groups the outcomes according to the 't' node, adds the 'HG' and 'AG' properties of the 'g' relationships, and aliases the outcomes as 'goals_scored' and 'goals_conceded,' accordingly. The results of the query are the name of the 't' node, the total number of goals scored by that team during the given season, and the total number of goals allowed by that team.

Output:

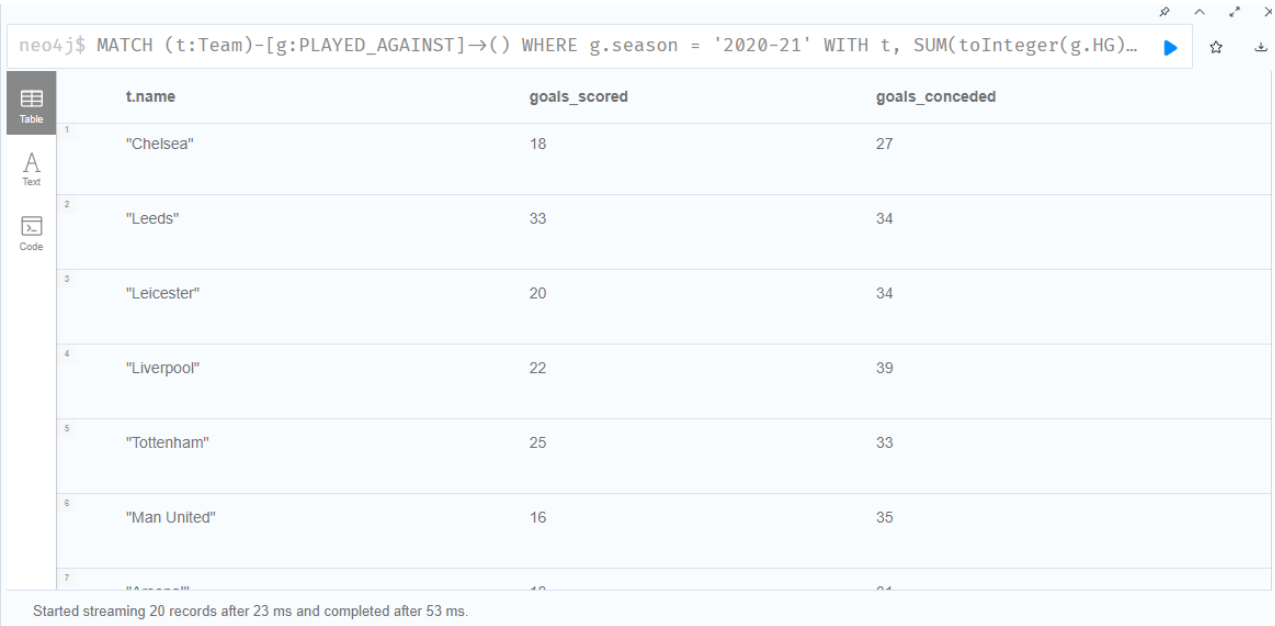

Fig.7. Total number of goals all the teams scored and conceded in the 2020/21 season.

5) Which team had the best home-winning record since 2000?

```
MATCH (ht:Team)-[g:PLAYED_AGAINST]->(at:Team)
WHERE g.season >= '2000' AND g.HG > g.AG
WITH ht, count(*) AS wins
RETURN ht.name AS team_name, sum(wins) AS total_home_wins
ORDER BY total_home_wins DESC
LIMIT 1
```

It matches the pattern where a team (ht) played against another team (at) through a relationship denoted as g, where the season is from 2000 onward, and the home team (ht) won the game. the number of victories for each home team (ht) in the result set is totaled. returns the name of the team with the most home victories overall since the year 2000, along with the number of those victories.

Output:

Fig.8. Team that had the best home-winning record since 2000.

6) Which team lost the most matches in the 2020/21 season?

```
MATCH (ht:Team)-[g:PLAYED_AGAINST]->(at:Team)
WHERE g.season = '2020-21' AND (g.Res = 'H' OR g.Res = 'A')
WITH at.name AS team_name, COUNT(CASE g.Res WHEN 'A' THEN 1 ELSE NULL END) AS losses,
COUNT(CASE g.Res WHEN 'H' THEN 1 ELSE NULL END) AS wins
ORDER BY losses DESC
LIMIT 1
RETURN team_name, losses
```

This search begins by matching the pattern of a Team node linked to another Team node via a PLAYED_AGAINST connection. The matches are then further filtered to only include those from the 2020–21 season and with a home win (H) or away win (A) as the outcome. The results are then organized by the away team's name (at.name), and conditional aggregation is used to calculate the number of wins and defeats. Depending on the value of the Res property, the CASE statement is utilized to filter out only the wins or losses. The team with the most losses along with the number of losses (team_name and losses) is returned after the results have been sorted by the number of losses in descending order.

Output:

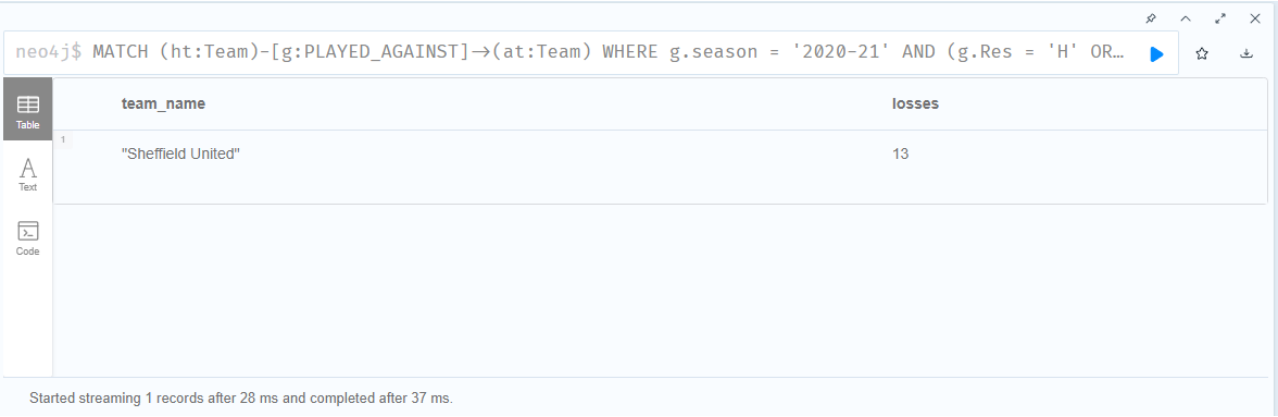

Fig.9. Team that lost the most matches in the 2020/21 season.

7) Which teams lost the 1st half but won the match in the 2020/21 season?

```
MATCH (ht:Team)-[g:PLAYED_AGAINST]->(at:Team)
WHERE g.season = '2020-21' AND g.HTR <> g.Res
AND ((g.HTR = 'H' AND g.Res = 'A') OR (g.HTR = 'A' AND g.Res = 'H'))
RETURN DISTINCT ht.name AS team_name
UNION
MATCH (ht:Team)<-[g:PLAYED_AGAINST]-(at:Team)
WHERE g.season = '2020-21' AND g.HTR <> g.Res
AND ((g.HTR = 'H' AND g.Res = 'A') OR (g.HTR = 'A' AND g.Res = 'H'))
RETURN DISTINCT at.name AS team_name
```

The first MATCH clause identifies teams that had a lead at halftime (g.HTR) but ultimately lost the game or tied it (g.Res). The HTR and Res values are checked to see if they reflect a change in the lead before

the WHERE clause filters for matches played in the 2020–21 season. Only distinct team names will be retrieved thanks to the DISTINCT keyword. Like the first MATCH clause, the second one identifies teams that were down at the break but went on to win or tie the game. By utilizing the DISTINCT keyword, it also only returns distinct team names. A list of all teams that match either criterion is returned by the UNION keyword, which combines the results of both MATCH clauses and eliminates duplicates.

Output:

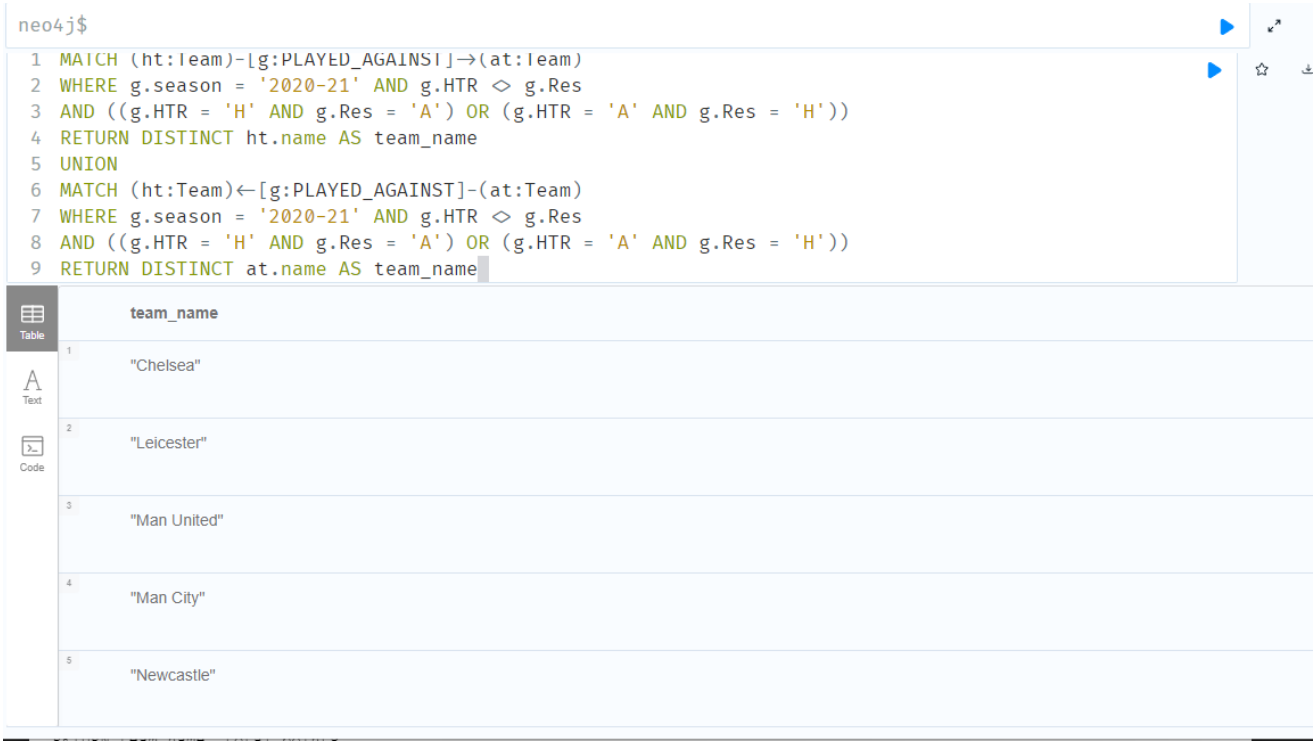

Fig.10. Teams that lost the 1st half but won the match in the 2020/21 season.

8) Which team earned the highest-ever points in all seasons since 2000?

```
MATCH (t:Team)-[g:PLAYED_AGAINST]->()
WHERE g.season >= '2000'
SET g.points = CASE g.Res
  WHEN 'H' THEN 3
  WHEN 'A' THEN 0
  WHEN 'D' THEN 1
END
WITH t.name AS team_name, SUM(g.points) AS total_points
ORDER BY total_points DESC
LIMIT 1
RETURN team_name, total_points
```

The points attribute of the PLAYED_AGAINST relationships is set based on the game outcome in this Cypher query, which matches all Team nodes that have participated in games in seasons after 2000. The CASE statement assigns 3 points for a home victory, 0 for a defeat away from home, and 1 for a tie. The query then aggregates the points for each team and groups the games by a given team. The result of the query is the name of the squad with the highest points overall across all contests played after 2000.

Output:

Fig.11. Team that earned the highest-ever points in all seasons since 2000.

9) Display the final league table ranking of all teams in the 2020/21 season (based on the total points).

```
MATCH (t:Team)-[g:PLAYED_AGAINST]->()
WHERE g.season = '2020-21'
WITH t.name AS team_name, SUM(g.points) AS total_points
ORDER BY total_points DESC
RETURN team_name, total_points
```

The points attribute of the PLAYED_AGAINST relationships is set based on the game outcome in this Cypher query, which matches all Team nodes that have participated in games in seasons after 2000. The CASE statement assigns 3 points for a home victory, 0 for a defeat away from home, and 1 for a tie. The query then organizes the matches by teams and totals the points for each team. The result of the query is the name of the squad with the highest points overall across all contests played after 2000.

Output:

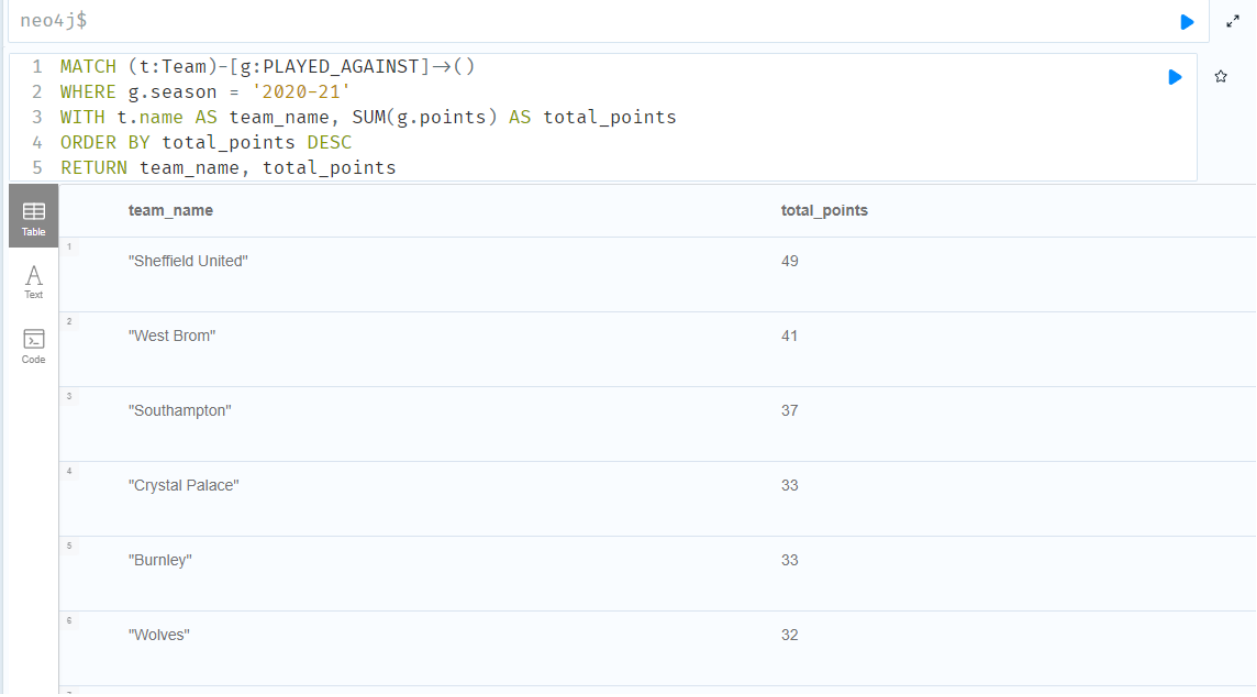

Fig.12. The final league table ranking of all teams in the 2020/21 season (based on the total points).

10) Which team had the longest unbeaten record?

```
MATCH (t:Team)-[g:PLAYED_AGAINST]->()
WHERE g.season >= '2000'
SET g.points = CASE g.Res
WHEN 'H' THEN 3
WHEN 'A' THEN 0
WHEN 'D' THEN 1
END
WITH t.name AS team_name, COLLECT(g) AS games
WITH team_name, games, REDUCE(unbeaten = true, g in games |
CASE g.Res
WHEN 'H' THEN NOT unbeaten
WHEN 'A' THEN unbeaten
WHEN 'D' THEN unbeaten
END
) AS unbeaten
```

```
WHERE unbeaten
RETURN team_name, size(games) AS unbeaten_record_length
ORDER BY unbeaten_record_length DESC
LIMIT 1
```

SET updates the relationship 'PLAYED_AGAINST' between nodes of type 'Team's' property 'points'. The squad that has gone the longest without losing since 2000 is then discovered. The team's points are determined using the relationship property 'points', which has been modified, based on the outcomes of each game: 3 points for a victory, 0 points for a defeat, and 1 point for a tie. The REDUCE function is then used to assess if the team was undefeated in each game after organizing the games by team and compiling them into a list for each team. The 'unbeaten' variable is set to true by the REDUCE function, which iterates over each game and updates it based on the outcome. If the team won at home, they are regarded as defeated; if they won away, they are still unbeaten. The team would still be unbeaten if the game ended in a tie. The name of the team and the duration of the unbeaten streak are returned if the team was unbeaten in every game on the list. The results are shown in descending order according to the length of the unbroken record.

Output:

Fig.13. Team that had the longest unbeaten record.